

```
0.9264411926269531),
('GREEN ENAMEL FLOWER RING',
0.9179285764694214),
('RED GLASS TASSLE BAG CHARM',
0.9122185707092285),
('BLUE GLASS CHUNKY CHARM BRACELET',
0.9082530736923218),
('AMBER DROP EARRINGS W LONG BEADS',
0.9068467617034912),
('GREEN MURANO TWIST BRACELET',
0.9062117338180542)]
```

The above findings are quite relevant and fit the input product well, but the output is only based on a single product's vector. Let's work on creating a system that will propose things based on the consumers' previous purchases. We'll do this by averaging all the vectors of the products the consumer has purchased so far, and using the resulting vector to discover related things. Create a function that takes a list of product IDs and returns a 100-dimensional vector, which is the mean of the product vectors in the input list.

```
def my_aggr_vec(products):
    p_vec = []
    for i in products:
        try:
            p_vec.append(model[i])
        except KeyError:
            continue

    return np.mean(p_vec, axis=0)

get_similar_item(my_aggr_vec(purchases_val[0]))
```

The output will be:

```
[('YELLOW DRAGONFLY HELICOPTER',
0.6904580593109131),
('JUMBO BAG RED RETROSPOT',
0.6636316776275635),
('CREAM HANGING HEART T-LIGHT
HOLDER', 0.6594936847686768),
('DISCOUNT', 0.6384902596473694),
('JUMBO BAG STRAWBERRY',
```

```
0.6319284439086914),
('WRAP 50'S CHRISTMAS",
0.6274842023849487)]

get_similar_item(my_aggr_vec(purchases_val[2]))

[('YELLOW DRAGONFLY HELICOPTER',
0.681573748588562),
('CREAM HANGING HEART T-LIGHT HOLDER',
0.6654664278030396),
('JUMBO BAG RED RETROSPOT',
0.6553223133087158),
('WOODEN FRAME ANTIQUE WHITE',
0.6327764391899109),
('DISCOUNT', 0.6313808560371399),
('JUMBO BAG STRAWBERRY',
0.6252866983413696)]
```

In the end, our system recommended six products based on a user's complete buying history. The model can also be used to make product suggestions based on recent purchases. Let's try it with only the previous ten purchases as input:

```
get_similar_item(my_aggr_vec(purchases_val[0][-10:]))
```

The output will be:

```
[('YELLOW DRAGONFLY HELICOPTER',
0.6946737170219421),
('JUMBO BAG RED RETROSPOT',
0.6675459742546082),
('CREAM HANGING HEART T-LIGHT HOLDER',
0.6548881530761719),
('DISCOUNT', 0.6417276859283447),
('WRAP 50'S CHRISTMAS",
0.6357795000076294),
('JUMBO BAG STRAWBERRY',
0.6348972320556641)]
```

## Summing it up

Word2vec is a fast and powerful method for learning a meaningful representation of words from context. In this article it has been used to solve a shopping-related problem. It enables us to precisely model each product using a vector of coordinates that captures the context in which the product was purchased. This allows us to quickly find products that are comparable for each user.

Suggesting the appropriate products/items to users can enhance not just interaction rates but also product purchases, resulting in increased income. Using machine learning to find similar products also raises product visibility. **END** 🐧

## References

- [1] Deep Learning for Natural Language Processing by Karthiek Reddy Bokka, Shubhangi Hora, Tanuj Jain, Monicah Wambugu, Packt
- [2] Natural Language Processing with Python by Steven Bird, Ewan Klein and Edward Loper, O'Reilly
- [3] Machine Learning for Text, 1st ed. 2018 Edition, by Charu C. Aggarwal, Springer Applied Text Analysis with Python by Benjamin Bengfort, Rebecca Bilbro, Tony Ojeda, O'Reilly
- [4] Python for Data Analysis: Data Wrangling with Pandas, NumPy, and IPython, by William McKinney, O'Reilly
- [5] <https://www.kaggle.com/code/dibyendubanerjee/product-recommendation-system-using-word2vec>
- [6] <https://pypi.org/project/gensim/>
- [7] <https://www.udemy.com/>

## By: Dibyendu Banerjee

The author is a senior architect at Cognizant's AI and analytics practice. He is a certified Deep Learning and NLP practitioner with overall 15+ years of IT experience. His area of expertise is in Python, R, Java, Google Cloud Platform and emerging open source technologies.